

Estructuras de Control

July 4, 2026

PAO25_25_03_Python_Control

July 01, 2026

PAO26-26 - Python 101, Estructuras de Control

Kevin Xavier Suasnavas Villarreal

[Link a mi GitHub](#)

1 Estructuras de control

1.1 Selección (sentencias condicionales)

- Selección de una de varias alternativas en base a alguna condición.
- Indentación para estructurar el código.
- Es importante utilizar el símbolo `:` para iniciar un bloque de instrucciones.

```
[153]: # Solicitar un valor entero al usuario
x = int(input("Ingrese el valor de x: "))

# Comprobar si el número es positivo
if x > 0:
    print("Valor positivo")
else:
    print("Valor no positivo")
```

Ingrese el valor de x: 8

Valor positivo

Las condiciones son expresiones que siempre se evalúan como **True** o **False**.

Para construir condiciones en Python se utilizan:

- Operadores de comparación.
- Operadores lógicos.
- Operadores de identidad.
- Operadores de pertenencia.

```
[155]: # Crear una variable entera y una lista
x = 3
y = [1, 2, 3]

# Comparar si x es mayor que 0
print(x > 0)

# Verificar si x está entre 0 y 10
print(x > 0 and x < 10)

# Comprobar que x y y no son el mismo objeto
print(x is not y)

# Verificar si x pertenece a la lista y
print(x in y)
```

True
True
True
True

Las sentencias condicionales pueden incluir más de dos alternativas mediante la palabra reservada **elif**, lo que permite evaluar varias condiciones en orden.

```
[156]: # Asignar un valor a la variable
x = -3

# Evaluar el valor utilizando varias condiciones
if x > 0:
    print("Valor positivo")
elif x == 0:
    print("Valor nulo")
else:
    print("Valor negativo")
```

Valor negativo

1.1.1 Anidamiento

Es posible colocar una estructura condicional dentro de otra para realizar evaluaciones más específicas.

```
[158]: # Asignar un valor a la variable
val = 120

# Evaluar primero si el valor es positivo
if val > 0:
    # Verificar si además es menor que 100
    if val < 100:
```

```

        print("Valor positivo")
    else:
        print("Valor muy positivo")
else:
    print("Valor negativo")

```

Valor muy positivo

2 2 Switch

A partir de **Python 3.10**, se incorporó la estructura **match-case**, que permite seleccionar entre múltiples alternativas de forma similar al **switch** de otros lenguajes de programación.

```

[159]: # Asignar un valor a la variable
a = -6

# Evaluar el valor usando match-case
match a:
    case _ if a > 0:
        print("positive")
    case _ if a == 0:
        print("zero")
    case _ if a < 0:
        print("negative")

```

negative

```

[160]: # Asignar una letra correspondiente a un día
a = "L"

# Evaluar el valor mediante match-case
match a:
    case "L":
        print("Lunes")
    case "M":
        print("Martes")
    case "X":
        print("Miércoles")
    case _:
        print("Wrong Day")

```

Lunes

2.1 Expresión ternaria

La **expresión ternaria** permite escribir una estructura **if-else** en una sola línea. Se recomienda utilizarla únicamente cuando la condición sea sencilla y fácil de entender.

```
[161]: # Asignar un número
x = -3

# Calcular el valor absoluto usando una expresión ternaria
resultado = x if x >= 0 else -x

# Mostrar el resultado
print(resultado)
```

3

```
[162]: # Asignar un número
val = -3

# Calcular el valor absoluto usando una estructura if-else
if val >= 0:
    resultado = val
else:
    resultado = -val

# Mostrar el resultado
print(resultado)
```

3

2.2 Concatenación de comparaciones

Es posible combinar varias condiciones dentro de una misma expresión utilizando operadores lógicos:

- **and**: devuelve `True` cuando ambas condiciones son verdaderas.
- **or**: devuelve `True` cuando al menos una condición es verdadera.
- **not**: invierte el valor lógico de una expresión.
- **elif**: permite evaluar una nueva condición cuando la anterior no se cumple.

2.2.1 Enlace a Tablas de Verdad

Los operadores lógicos pueden combinar varias condiciones dentro de una misma estructura condicional para tomar decisiones más específicas.

```
[163]: # Definir el género y el año de estreno de una película
genero = "Drama"
fecha_de_estreno = 1989

# Evaluar diferentes condiciones
if genero == "Comedia" or genero == "Acción":
    print("¡Buena película!")
elif fecha_de_estreno >= 1990 and fecha_de_estreno < 2000:
    print("¡Buena década!")
else:
```

```
print("Meh")
```

Meh

2.3 Comparación de variables: == vs is

Python dispone de dos formas de comparar variables:

- **==**: compara si los **valores** de dos objetos son iguales.
- **is**: compara si ambas variables hacen referencia al **mismo objeto en memoria**.

```
[164]: # Crear dos variables con el mismo valor
a = 1000
b = a

# Comparar identidad y valor
print(a is b)
print(a == b)

# Obtener el tipo de cada variable
c = type(a)
print(c)

d = type(b)
print(d)

# Comparar los tipos
print(c == d)

# Mostrar el identificador de cada objeto
print(id(a))
print(id(b))
```

True

True

<class 'int'>

<class 'int'>

True

2965583830288

2965583830288

Nota: En el PDF aparecen resultados donde `a is b` y `a == b` muestran `False` y los tipos son diferentes (`int` y `float`). Ese comportamiento **no corresponde al código mostrado**. Al ejecutar exactamente este código, `b = a` hace que ambas variables referencien el mismo objeto, por lo que el resultado esperado es el mostrado arriba.

```
[165]: # Crear dos listas con el mismo contenido
a = [1, 2, 3]
b = [1, 2, 3]
```

```

# Hacer que c haga referencia a la misma lista que a
c = a

# Comparar identidad entre los objetos
print(a is b)
print(a is c)

# Comparar si los valores son iguales
print(a == b)

# Mostrar el identificador de cada objeto
print(id(a))
print(id(b))
print(id(c))

```

```

False
True
True
2965582969216
2965583849856
2965582969216

```

2.4 Valor booleano

Todos los objetos en Python poseen de forma implícita un valor booleano (True o False).

- Cualquier número diferente de **0** tiene el valor **True**.
- Cualquier colección que contenga elementos también tiene el valor **True**.
- El número **0**, las colecciones vacías y el objeto especial **None** tienen el valor **False**.

```

[166]: # Asignar valores a las variables
a = -10      # Se considera True
b = None     # Se considera False

# Verificar el valor booleano de las variables
if a:
    print("a es True")

if not b:
    print("b es False")

```

```

a es True
b es False

```

```

[167]: # Crear una lista con elementos
c = [1, 2, 3]

# Reasignar la variable a None

```

```

c = None

# Comprobar si la variable contiene un valor
if c:
    print("Lista no vacía")
else:
    print("Lista vacía")

```

Lista vacía

3 Iteración (bucles)

Los **bucles** permiten repetir un bloque de instrucciones varias veces hasta que se cumpla una condición determinada.

Existen dos tipos principales de bucles en Python:

- **while**: ejecuta un bloque mientras una condición sea verdadera.
- **for**: recorre los elementos de una secuencia o colección.

2.4.1 Bucle while

El bucle **while** ejecuta un conjunto de instrucciones **mientras una condición sea True**.

Características:

- La condición se evalúa antes de cada iteración.
- Si la condición es falsa desde el inicio, el bloque nunca se ejecuta.
- Es importante actualizar la condición para evitar bucles infinitos.

Sintaxis general:

“python while condicion: instrucciones

```

[170]: ### **Code 1 - Mostrar los primeros tres elementos de una lista**

# Inicializo el índice desde la primera posición de la lista.
indice = 0

# Creo una lista con varios números.
numeros = [9, 4, 7, 1, 2]

# Recorro únicamente las tres primeras posiciones.
while indice < 3:
    print(numeros[indice])
    indice += 1

```

9

4

7

```
[171]: # Creo una lista con varios números.
numeros = [33, 3, 9, 21, 1, 7, 12, 10, 8]

# Inicializo el contador y el índice.
contador = 0
indice = 0

# Recorro toda la lista.
while indice < len(numeros):

    # Verifico si el número es menor que 10.
    if numeros[indice] < 10:
        print(numeros[indice])
        contador += 1

    # Avanzo a la siguiente posición.
    indice += 1

# Muestro la cantidad de números encontrados y el índice final.
print("Contador:", contador)
print("Índice:", indice)
```

```
3
9
1
7
8
Contador: 5
Índice: 9
```

2.5 3 Iteración (bucles)

Los **bucles** permiten repetir un bloque de código varias veces. Python dispone de dos estructuras principales:

- **while:** repite mientras una condición sea verdadera.
- **for:** recorre los elementos de una colección u objeto iterable.

2.5.1 Bucle while

El bucle **while** ejecuta un bloque de instrucciones mientras una condición sea verdadera.

- Si la condición es falsa desde el inicio, el bloque nunca se ejecuta.
- Es importante actualizar la condición para evitar bucles infinitos.

Formato general:

“python while condicion: instrucciones


```
[172]: ### **Code**

# Recorro los tres primeros elementos de una lista usando un índice

indice = 0
numeros = [9, 4, 7, 1, 2]

while indice < 3:
    print(numeros[indice])
    indice += 1
```

9
4
7

```
[173]: # Recorro una lista y cuento cuántos números son menores que 10

numeros = [33, 3, 9, 21, 1, 7, 12, 10, 8]
contador = 0
indice = 0

while indice < len(numeros):
    if numeros[indice] < 10:
        print(numeros[indice])
        contador += 1
    indice += 1

print("Contador:", contador)
print("Índice:", indice)
```

3
9
1
7
8
Contador: 5
Índice: 9

```
[174]: # Voy eliminando el primer carácter del texto en cada iteración

nombre = "Pablo"

while nombre:
    print(nombre)
    nombre = nombre[1:]
```

Pablo
ablo
blo

lo
o

[175]: *# Muestro los números del 0 al 9 utilizando un bucle while*

```
i = 0

while i < 10:
    print(i)
    i += 1
```

0
1
2
3
4
5
6
7
8
9

2.5.2 Bucle for

El bucle **for** permite recorrer automáticamente los elementos de una secuencia u objeto iterable como listas, tuplas, cadenas de texto o rangos.

Formato general:

“python for elemento in objeto: instrucciones

[176]: *### **Code***

```
# Recorro una lista de películas e imprimo cada elemento

peliculas = ["Matrix", "The Purge", "Avatar", "Star Wars"]

for pelicula in peliculas:
    print(pelicula)
```

Matrix
The Purge
Avatar
Star Wars

2.5.3 También se usa para iterar un número preestablecido de veces (*counted loops*)

El bucle **for** también puede utilizarse para repetir un bloque de instrucciones una cantidad determinada de veces mediante la función **range()**.

```
[177]: # Recorro los números del 0 al 9 utilizando range()  
for i in range(10):  
    print(i)
```

0
1
2
3
4
5
6
7
8
9

2.5.4 range() es útil en combinación con len()

La función `len()` devuelve el número de elementos de una secuencia. Al combinarla con `range()` podemos recorrer una colección utilizando sus índices para acceder a cada elemento por posición.

```
[178]: # Recorro un string utilizando sus índices  
nombre = "Pablo"  
  
for i in range(len(nombre)):  
    print(nombre[i])
```

P
a
b
l
o

También es posible recorrer directamente cada elemento de una secuencia sin utilizar índices, lo que hace el código más sencillo cuando no necesitamos conocer la posición de cada elemento.

```
[179]: # Recorro directamente cada carácter del string  
nombre = "Pablo"  
  
for letra in nombre:  
    print(letra)
```

P
a
b
l
o

2.5.5 Iteración de tuplas

Las tuplas también pueden recorrerse con un bucle `for`. Si cada tupla tiene la misma cantidad de elementos, es posible desempaquetar sus valores directamente en varias variables.

```
[180]: # Recorro una lista de tuplas desempaquetando sus elementos
tuplas = [(1, 2, 4), (3, 4), (5, 6)]

for a, b, c in tuplas:
    print(a, b, c)
```

```
1 2 4
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[180], line 4
      1 # Recorro una lista de tuplas desempaquetando sus elementos
      2 tuplas = [(1, 2, 4), (3, 4), (5, 6)]
      3
----> 4 for a, b, c in tuplas:
      5     print(a, b, c)

ValueError: not enough values to unpack (expected 3, got 2)
```

2.5.6 Iteración de diccionarios

Al recorrer un diccionario con un bucle `for`, por defecto se obtienen únicamente las claves.

```
[181]: # Recorro un diccionario obteniendo sus claves
diccionario = {"a": 1, "b": 2, "c": 3}

for key in diccionario:
    print(f"{key} => {diccionario[key]}")
```

```
a => 1
b => 2
c => 3
```

Para recorrer simultáneamente las claves y los valores se utiliza el método `items()`. Si solo se necesitan los valores, se emplea el método `values()`.

```
[182]: # Recorro el diccionario obteniendo la clave y el valor
for key, value in diccionario.items():
    print(f"{key} => {value}")
```

```
a => 1
b => 2
c => 3
```

```
[183]: # Recorro únicamente los valores del diccionario
for value in diccionario.values():
    print(value)
```

```
1
2
3
```

La variable utilizada en la cabecera del `for` puede ser cualquier expresión válida para una asignación. Esto permite desempaquetar directamente los elementos de una colección.

```
[184]: # Desempaquetar directamente los elementos de cada tupla
for a, b, c in [(1, 2, 3), (4, 5, 6)]:
    print(a, b, c)
```

```
1 2 3
4 5 6
```

2.6 3.1 Las sentencias `break`, `continue` y `else`

2.6.1 `break` y `continue`

Las sentencias `break` y `continue` únicamente tienen sentido dentro de un bucle.

- `break` finaliza inmediatamente la ejecución del bucle.
- `continue` omite el resto de la iteración actual y continúa con la siguiente.

Generalmente se utilizan dentro de estructuras condicionales (`if`) para controlar el flujo de ejecución.

```
[185]: # Busco una calificación específica y detengo el recorrido cuando la encuentro
rating_to_find = 4.2
movie_ratings = [4.3, 2.5, 1.7, 4.2, 3.8, 3.3, 4.5]

for rating in movie_ratings:
    print(rating)
    if rating == rating_to_find:
        print("Found")
        break
```

```
4.3
2.5
1.7
4.2
Found
```

```
[186]: # Muestro únicamente los números impares utilizando continue
for i in range(10):
    if i % 2 == 0:
        continue # Si el número es par, paso a la siguiente iteración
```

```
print(f"Odd number {i}")
```

```
Odd number 1
Odd number 3
Odd number 5
Odd number 7
Odd number 9
```

La sentencia `continue` ayuda a reducir el número de niveles de anidamiento, haciendo que el código sea más claro y fácil de leer.

Sin utilizar `continue`, el ejemplo anterior puede escribirse de la siguiente manera:

```
[187]: # Muestro los números impares sin utilizar continue
for i in range(10):
    if i % 2 != 0:
        print("Numero impar:", end=" ")
        print(i)
```

```
Numero impar: 1
Numero impar: 3
Numero impar: 5
Numero impar: 7
Numero impar: 9
```

```
[188]: # Recorro dos bucles anidados y utilizo break para salir del bucle interno
for i in range(5):
    for a in range(2):
        print(f"{i} - {a}")
        if i == 3 and a == 0:
            break
```

```
0 - 0
0 - 1
1 - 0
1 - 1
2 - 0
2 - 1
3 - 0
4 - 0
4 - 1
```

2.6.2 else

Los bucles en Python pueden incluir una sentencia `else`.

Esta característica puede resultar poco intuitiva, ya que no existe en muchos otros lenguajes de programación.

El bloque `else` se ejecuta únicamente cuando el bucle finaliza de forma normal, es decir, cuando no termina mediante una sentencia `break`.

```
[189]: # Busco un elemento en la lista utilizando for-else
lista = [60, 60, 30, 60]
element_to_find = 60

for element in lista:
    if element == element_to_find:
        print("found")
        break
else:
    print("not found")
```

found

```
[190]: # Busco un elemento utilizando una variable bandera (flag)
lista = [10, 30, 50, 40]
element_to_find = 30
found = False

for element in lista:
    if element == element_to_find:
        found = True
        break

if found:
    print("dato encontrado")
else:
    print("not found")
```

dato encontrado

3 4 List Comprehensions

Las **List Comprehensions** permiten crear listas de una forma más compacta utilizando un bucle for.

Siempre se escriben entre corchetes (`[]`), ya que su resultado es una nueva lista.

3.0.1 Sintaxis

```
[expresion for item in iterable]
```

En cada iteración se evalúa la expresión y su resultado se añade automáticamente a la nueva lista.

3.0.2 Ejemplo: repetir los caracteres de un string

```
[191]: # Repito tres veces cada carácter de un string
[char * 3 for char in "Enrique"]
```

```
[191]: ['EEE', 'nnn', 'rrr', 'iii', 'qqq', 'uuu', 'eee']
```

```
[192]: # Construyo la misma lista utilizando un bucle for tradicional
lista = []

for char in "Enrique":
    lista.append(char)

print(lista)
```

```
['E', 'n', 'r', 'i', 'q', 'u', 'e']
```

El elemento utilizado en el `for` normalmente aparece en la expresión principal, aunque esto no es obligatorio. También es posible generar un mismo valor en cada iteración.

```
[193]: # Genero una lista con el mismo valor para cada elemento del iterable
[2 for _ in "Enrique"]
```

```
[193]: [2, 2, 2, 2, 2, 2, 2]
```

3.0.3 Versión extendida

Las **List Comprehensions** también permiten incluir un filtro mediante una condición `if`, de forma que solo se agreguen los elementos que la cumplan.

3.0.4 Sintaxis

```
[expresion for item in iterable if condicion]
```

3.0.5 Ejemplo: obtener números pares

```
[194]: # Obtengo únicamente los números pares utilizando una List Comprehension
[y for y in range(9) if y % 2 == 0]
```

```
[194]: [0, 2, 4, 6, 8]
```

Las **List Comprehensions** no son obligatorias. El mismo resultado puede obtenerse utilizando un bucle `for` tradicional.

```
[195]: # Construyo la lista de números pares utilizando un for tradicional
pares = []

for x in range(9):
    if x % 2 == 0:
        pares.append(x)

print(pares)
```

```
[0, 2, 4, 6, 8]
```

3.0.6 Versión completa

Una **List Comprehension** también puede incluir una expresión alternativa utilizando `if` y `else`.

3.0.7 Sintaxis

```
[expresion_1 if condicion else expresion_2 for item in iterable]
```

3.0.8 Ejemplo: reemplazar los números pares por cero

```
[196]: # Reemplazo por cero los números pares  
[0 if x % 2 == 0 else x for x in range(9)]
```

```
[196]: [0, 1, 0, 3, 0, 5, 0, 7, 0]
```

```
[197]: # Elevo al cuadrado los números mayores que 2  
[x**2 if x > 2 else x for x in range(-4, 5) if x > 0]
```

```
[197]: [1, 2, 9, 16]
```

```
[198]: # Obtengo el mismo resultado utilizando un bucle for tradicional  
lista = []  
  
for x in range(-4, 5):  
    if x > 0:  
        if x > 2:  
            lista.append(x**2)  
        else:  
            lista.append(x)  
  
print(lista)
```

```
[1, 2, 9, 16]
```

```
[199]: # Utilizo múltiples condiciones dentro de una List Comprehension  
[x**2 if x > 0 else 100 if x == 0 else x for x in range(-4, 5)]
```

```
[199]: [-4, -3, -2, -1, 100, 1, 4, 9, 16]
```

3.1 4.0.2 Versión completa

La sentencia `if` de una **List Comprehension** también puede incluir una expresión alternativa mediante `else`.

Sintaxis:

```
[expresion1 if condicion else expresion2 for elemento in iterable]
```

Esto permite decidir qué valor agregar a la lista dependiendo de una condición.

Ejemplo: reemplazar los números pares por 0.

```
[200]: # Recorro los números del 0 al 8  
# Si el número es par agrego 0, caso contrario agrego el mismo número  
resultado = [0 if x % 2 == 0 else x for x in range(9)]
```

```
# Muestro la lista resultante
print(resultado)
```

[0, 1, 0, 3, 0, 5, 0, 7, 0]

También es posible combinar una condición dentro del `if` y un filtro al final de la comprensión.

En este ejemplo solo se procesan los números positivos.

```
[201]: # Recorro los números de -4 a 4
# Solo tomo los positivos
# Si el número es mayor que 2 lo elevo al cuadrado
# Caso contrario dejo el número igual
resultado = [x**2 if x > 2 else x for x in range(-4, 5) if x > 0]

# Muestro el resultado
print(resultado)
```

[1, 2, 9, 16]

El mismo resultado también puede obtenerse utilizando un bucle `for` tradicional.

```
[202]: # Creo una lista vacía
lista = []

# Recorro los números de -4 a 4
for x in range(-4, 5):

    # Solo proceso los números positivos
    if x > 0:

        # Si es mayor que 2 agrego su cuadrado
        if x > 2:
            lista.append(x**2)

        # Caso contrario agrego el número
        else:
            lista.append(x)

# Muestro la lista
print(lista)
```

[1, 2, 9, 16]

También pueden encadenarse varias condiciones utilizando más de un `if` y `else`.

```
[203]: # Recorro los números de -4 a 4
# Si es positivo lo elevo al cuadrado
# Si es cero agrego 100
# Si es negativo dejo el valor original
```

```

resultado = [x**2 if x > 0 else 100 if x == 0 else x for x in range(-4, 5)]

# Muestro el resultado
print(resultado)

```

[-4, -3, -2, -1, 100, 1, 4, 9, 16]

El mismo resultado anterior puede escribirse utilizando un **for** con varias condiciones.

```

[205]: # Creo una lista vacía
lista = []

# Recorro los números de -4 a 4
for x in range(-4, 5):

    # Si el número es positivo agrego su cuadrado
    if x > 0:
        lista.append(x**2)

    # Si el número es cero agrego 100
    elif x == 0:
        lista.append(100)

    # Si es negativo agrego el mismo número
    else:
        lista.append(x)

# Muestro la lista final
print(lista)

```

[-4, -3, -2, -1, 100, 1, 4, 9, 16]

3.2 4.0.3 Anidamiento

Las **List Comprehensions** también permiten utilizar bucles anidados.

En este ejemplo se generan todas las combinaciones posibles entre dos listas y el resultado se almacena como una lista de tuplas.

```

[206]: # Defino las listas
lista_1 = [1, 2, 3]
lista_2 = [4, 5]

# Genero todas las combinaciones entre ambas listas
resultado = [(x, y) for x in lista_1 for y in lista_2]

# Muestro el resultado
print(resultado)

```

[(1, 4), (1, 5), (2, 4), (2, 5), (3, 4), (3, 5)]

3.3 4.0.4 Pros y contras

3.3.1 Ventajas

- Código más corto y limpio.
- Mayor legibilidad cuando la expresión es sencilla.
- Mejor rendimiento que un bucle tradicional en muchos casos.

3.3.2 Desventajas

- Si la comprensión contiene muchas condiciones o varios niveles de anidamiento, el código puede ser difícil de leer y mantener.

3.4 4.0.5 Otros tipos de comprehensions

3.4.1 Dictionary Comprehensions

Utilizando llaves {} es posible crear diccionarios de forma compacta.

La sintaxis general es:

```
{clave: valor for elemento in iterable}
```

En este ejemplo cada número será la clave y su cuadrado será el valor.

```
[207]: # Creo un diccionario donde la clave es el número
# y el valor es su cuadrado
d = {x: x * x for x in range(10)}

# Muestro el diccionario
print(d)

# Muestro el tipo de dato
print(type(d))
```

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
<class 'dict'>
```

3.5 Dictionary Comprehensions

También es posible crear diccionarios a partir de dos listas utilizando la función `zip()`.

En este ejemplo, una lista contiene los nombres de productos y otra sus precios. Ambos se unen para formar un diccionario.

```
[208]: # Defino la lista de productos
items = ['Banana', 'Pear', 'Olives']

# Defino la lista de precios
price = [1.1, 1.4, 2.4]

# Creo un diccionario uniendo ambas listas con zip()
shopping = {k: v for k, v in zip(items, price)}
```

```
# Muestro el diccionario
print(shopping)
```

```
{'Banana': 1.1, 'Pear': 1.4, 'Olives': 2.4}
```

La función `zip()` devuelve un objeto iterable formado por tuplas. Cada tupla contiene un elemento de cada colección.

```
[209]: # Defino las listas
items = ['Banana', 'Pear', 'Olives']
price = [1.1, 1.4, 2.4]

# Recorro las tuplas generadas por zip()
for k in zip(items, price):

    # Muestro el tipo de dato
    print(type(k))

    # Muestro el contenido de la tupla
    print(k)
```

```
<class 'tuple'>
('Banana', 1.1)
<class 'tuple'>
('Pear', 1.4)
<class 'tuple'>
('Olives', 2.4)
```

3.6 Set Comprehensions

Utilizando llaves `{}` y una expresión similar a las **List Comprehensions**, es posible crear conjuntos (`set`).

En este ejemplo se genera un conjunto con los diez primeros números naturales.

```
[210]: # Creo un conjunto con los números del 0 al 9
c = {x for x in range(10)}

# Muestro el conjunto
print(c)

# Muestro el tipo de dato
print(type(c))
```

```
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
<class 'set'>
```

3.7 Tuple Comprehensions

Las tuplas no poseen una comprensión propia como listas o conjuntos.

Sin embargo, es posible generar una secuencia con una expresión generadora y convertirla posteriormente en una tupla mediante la función `tuple()`.

```
[211]: # Creo una tupla utilizando una expresión generadora
tupla = tuple(x for x in range(4))

# Muestro la tupla
print(tupla)
```

(0, 1, 2, 3)

4 5 Excepciones

Las **excepciones** son eventos que representan situaciones inesperadas o errores durante la ejecución de un programa.

Cuando ocurre una excepción, Python interrumpe el flujo normal del programa.

Las excepciones permiten:

- Detectar errores automáticamente.
- Evitar que el programa finalice de forma inesperada.
- Ejecutar acciones alternativas cuando ocurre un error.

El mecanismo más utilizado para manejar excepciones es la estructura **try / except**.

4.1 Try / Except

Permite intentar ejecutar un bloque de código y capturar una excepción si ocurre un error.

Ejemplo: acceso a una posición inexistente de una lista.

4.1.1 Ejemplo sin manejo de excepciones

Si ocurre un error y no existe un bloque para manejarlo, la ejecución del programa se detiene inmediatamente. En este ejemplo se intenta acceder a una posición que no existe en la lista.

```
[212]: # Creo una lista con cuatro elementos
lista = [6, 1, 0, 5]

# Intento acceder a una posición inexistente
lista[4]

# Esta línea nunca se ejecutará debido al error anterior
print("Código tras el error")
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[212], line 5
      1 # Creo una lista con cuatro elementos
      2 lista = [6, 1, 0, 5]
```

```

3
4 # Intento acceder a una posición inexistente
----> 5 lista[4]
6
7 # Esta línea nunca se ejecutará debido al error anterior
8 print("Código tras el error")

```

IndexError: list index out of range

4.1.2 Capturar una excepción específica

Mediante `try` y `except` es posible controlar un error específico y continuar la ejecución del programa.

```

[214]: # Creo una lista
lista = [6, 1, 0, 5]

# Defino un índice fuera del rango de la lista
i = 300

try:
    # Intento acceder a una posición inexistente
    a = lista[i]

except IndexError:
    # Capturo el error de índice
    print("He capturado la excepción de tipo IndexError")
    a = 0

# El programa continúa ejecutándose
print("Código tras el bloque try")
print(a)

```

He capturado la excepción de tipo `IndexError`

Código tras el bloque `try`

0

Normalmente se recomienda capturar el tipo de excepción más específico posible. Sin embargo, también es posible capturar cualquier excepción utilizando únicamente `except`.

```

[215]: # Intento realizar una división por cero
try:
    4 / 0

# Capturo cualquier excepción
except:
    print("He capturado la división por cero")

```

He capturado la división por cero

También es posible guardar la excepción en una variable para mostrar el mensaje de error generado por Python.

```
[216]: # Creo una lista
lista = [6, 1, 0, 5]

try:
    # Intento acceder a una posición inexistente
    print(lista[4])

except Exception as e:
    # Muestro el mensaje de la excepción
    print("Error = " + str(e))

# El programa continúa ejecutándose
print("Reachable Code")
```

```
Error = list index out of range
Reachable Code
```

4.2 Try / Finally

Además del bloque `except`, Python ofrece la cláusula `finally`.

El código ubicado dentro de `finally` **siempre se ejecuta**, ocurra o no una excepción.

Esta estructura se utiliza principalmente para liberar recursos, como cerrar archivos, conexiones a bases de datos o liberar memoria utilizada por el programa.

4.2.1 Ejemplo de try con finally

El bloque `finally` se ejecuta siempre, independientemente de que ocurra o no una excepción.

En este ejemplo no se produce ningún error, pero el bloque `finally` se ejecuta antes de continuar con el resto del programa.

```
[217]: # Creo una lista
lista = [6, 1, 0, 5]

try:
    # Accedo a una posición válida de la lista
    a = lista[1]

except IndexError:
    # Este bloque solo se ejecutaría si ocurre un IndexError
    print("Exception IndexError Captured")

finally:
    # Este bloque siempre se ejecuta
    print("Bloque finally")
```



```
# Continúo ejecutando el programa
b = lista[2]
print("Código tras el bloque try")
print(b)
```

Bloque finally
Código tras el bloque try
0

4.3 raise

La sentencia **raise** permite lanzar excepciones de forma explícita.

Se utiliza cuando el programador desea generar un error de manera intencional para indicar que una condición no es válida o que ocurrió una situación excepcional.

```
[218]: # Creo una lista
lista = [6, 1, 0, 5]

# Defino un índice fuera del rango
indice = 4

try:
    # Verifico si el índice es válido
    if indice >= len(lista):

        # Lanzo una excepción de forma explícita
        raise IndexError("Índice fuera de rango")

except IndexError:
    # Capturo la excepción lanzada
    print("Excepción de tipo IndexError capturada")
```

Excepción de tipo IndexError capturada

4.4 5.1 Ejercicios

En esta sección se presentan ejercicios para practicar las estructuras de control en Python, como sentencias condicionales, bucles, manejo de excepciones y comprehensions. Los ejercicios permiten reforzar el uso de listas, diccionarios, conjuntos, tuplas y técnicas de programación fundamentales.

1. Escribe un programa que calcule la suma de todos los elementos de una lista dada. La lista sólo puede contener elementos numéricos.
2. Dada una lista con elementos duplicados, escribir un programa que muestre una nueva lista con el mismo contenido que la primera pero sin elementos duplicados. Para este ejercicio, no puedes hacer uso de objetos de tipo **set**.
3. Escribe un programa que construya un diccionario que contenga un número (entre 1 y n) de elementos de esta forma: (x, x*x). Ejemplo: para n = 5, el diccionario resultante sería

{1:1, 2:4, 3:9, 4:16, 5:25}.

4. Escribe un programa que, dada una lista de palabras, compruebe si alguna empieza por 'a' y tiene más de 9 caracteres. Si dicha palabra existe, el programa deberá terminar en el momento exacto de encontrarla. Además, deberá mostrar un mensaje indicando si la búsqueda tuvo éxito o no y, en caso afirmativo, la palabra encontrada.
5. Dada una lista L de números positivos, escribir un programa que muestre otra lista (ordenada) con los índices i donde L[i] sea múltiplo de 3.
6. Dado un diccionario cuyas claves son cadenas (**str**) y cuyos valores son números (**int** o **float**), escribe un programa que muestre la clave cuyo valor asociado sea el mayor de todo el diccionario.
7. Dadas las listas **a** = [2, 4, 6, 8] y **b** = [7, 11, 15, 22], escribe un programa que multiplique cada elemento de **a** mayor que 5 por cada elemento de **b** menor que 14 y muestre los resultados.
8. Escribe un programa que solicite al usuario un número X mediante **input()** y muestre el resultado de 10 / X. El programa debe manejar correctamente posibles excepciones, como división entre cero o entradas no numéricas.
9. Escribe un programa que cree un diccionario y solicite al usuario una clave mediante **input()**. Si la clave existe, deberá mostrar su valor asociado; en caso contrario, deberá gestionar el error mostrando un mensaje informativo.
10. Escribe una **list comprehension** que construya una lista con los números enteros positivos de una lista dada, descartando los valores de tipo **float**.
11. Escribe una **set comprehension** que, dada una palabra, construya un conjunto con las vocales presentes en dicha palabra.
12. Escribe una **list comprehension** que construya una lista con todos los números del 0 al 50 que contengan el dígito 3. El resultado esperado es: [3, 13, 23, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 43].
13. Escribe una **dictionary comprehension** que construya un diccionario con cada palabra de una frase y su longitud. Por ejemplo, para "Soy un ser humano" el resultado será: {'Soy': 3, 'un': 2, 'ser': 3, 'humano': 6}.
14. Escribe una **list comprehension** que construya una lista con los números del 1 al 10, donde la primera mitad aparezca como números y la segunda mitad como texto. El resultado esperado es: [1, 2, 3, 4, 5, 'seis', 'siete', 'ocho', 'nueve', 'diez'].

5 6 Soluciones

A continuación se presentan las soluciones propuestas para los ejercicios del capítulo anterior. Cada solución aplica los conceptos estudiados sobre estructuras de control, listas, diccionarios, bucles, manejo de excepciones y comprehensions en Python.

5.1 Ejercicio 1

Calcular la suma de todos los elementos de una lista de números.

```
[219]: # Defino una lista de números
numeros = [1, 5, 9, 2, 3]

# Inicializo la suma
suma = 0

# Recorro la lista y acumulo los valores
for numero in numeros:
    suma += numero

# Muestro la suma total
print(suma)
```

20

5.2 Ejercicio 2

Eliminar los elementos duplicados de una lista sin utilizar conjuntos (`set`).

```
[220]: # Defino una lista con elementos repetidos
numeros = [1, 2, 2, 3, 4, 5, 4, 5, 3, 3, 1]

# Creo una lista vacía para el resultado
resultado = []

# Recorro la lista original
for numero in numeros:

    # Agrego solo los elementos que aún no existen
    if numero not in resultado:
        resultado.append(numero)

# Muestro la nueva lista
print(resultado)
```

[1, 2, 3, 4, 5]

5.3 Ejercicio 3

Construir un diccionario donde cada clave sea un número del 1 al `n` y su valor sea el cuadrado de dicho número.

```
[221]: # Defino el valor máximo
n = 5

# Creo un diccionario vacío
diccionario = {}

# Lleno el diccionario con los cuadrados
```

```

for i in range(1, n + 1):
    diccionario[i] = i * i

# Muestro el diccionario
print(diccionario)

```

{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

5.4 Ejercicio 4

Buscar la primera palabra que comience con la letra “a” y tenga nueve o más caracteres. Si se encuentra, el programa termina inmediatamente.

```

[222]: # Defino una lista de palabras
palabras = [
    "",
    "Valencia",
    "python",
    "asignaturas",
    "programación",
    "java",
    "estudiantes",
    "académicos"
]

# Recorro la lista
for palabra in palabras:

    # Verifico la condición
    if len(palabra) >= 9 and palabra[0] == 'a':
        print(f"Palabra encontrada: {palabra}")
        break

    else:
        # Se ejecuta si no se encontró ninguna palabra
        print("Palabra no encontrada")

```

Palabra encontrada: asignaturas

5.5 Ejercicio 5

Obtener una lista con los índices cuyos elementos son múltiplos de 3.

```

[223]: # Defino la lista de números
L = [3, 5, 13, 12, 1, 9]

# Creo una lista para guardar los índices
resultado = []

```

```

# Recorro la lista por posiciones
for i in range(len(L)):

    # Verifico si el elemento es múltiplo de 3
    if L[i] % 3 == 0:
        resultado.append(i)

# Muestro los índices encontrados
print(resultado)

```

[0, 3, 5]

5.6 Ejercicio 6

Encontrar la clave cuyo valor asociado sea el mayor dentro de un diccionario.

```

[224]: # Defino un diccionario
d = {'a': 4.3, 'b': 1, 'c': 7.8, 'd': -5}

# Inicializo el valor máximo
maximo = float("-inf")

# Verifico si el diccionario está vacío
if len(d) == 0:
    print("El diccionario está vacío")

# Recorro el diccionario
for clave, valor in d.items():

    # Actualizo el máximo si encuentro un valor mayor
    if valor > maximo:
        maximo = valor
        resultado = clave

# Muestro la clave encontrada
print(resultado)

```

c

5.7 Ejercicio 7

Multiplicar los elementos de la lista a mayores que 5 por los elementos de la lista b menores que 14.

```

[225]: # Defino las listas
a = [2, 4, 6, 8]
b = [7, 11, 15, 22]

# Recorro la primera lista

```

```

for i in a:

    # Verifico que el elemento sea mayor que 5
    if i > 5:

        # Recorro la segunda lista
        for j in b:

            # Verifico que el elemento sea menor que 14
            if j < 14:
                print(f"{i} x {j} = {i*j}")

```

```

6 x 7 = 42
6 x 11 = 66
8 x 7 = 56
8 x 11 = 88

```

5.8 Ejercicio 8

Solicitar un número al usuario y calcular la división $10 / X$, manejando posibles excepciones.

```

[226]: try:
    # Solicito un valor al usuario
    x = input()

    # Convierto el valor a número
    numero = float(x)

    # Muestro el resultado de la división
    print(f"El resultado de 10/{x} es: {10/numero}")

except ValueError:
    # Capturo un valor no numérico
    print(f"El valor '{x}' no es numérico")

except ZeroDivisionError:
    # Capturo la división por cero
    print("Error: división por cero")

```

8

El resultado de 10/8 es: 1.25

5.9 Ejercicio 9

Solicitar una clave de un diccionario y mostrar su valor asociado, controlando el error si la clave no existe.

```
[227]: # Defino un diccionario
notas = {
    'Programación': 9.5,
    'Inteligencia artificial': 7.6,
    'Redes': 5.2,
    'Sistemas operativos': 6.4
}

try:
    # Solicito una asignatura
    asignatura = input("Introduce una asignatura: ")

    # Muestro la nota correspondiente
    print(f"La nota de la asignatura '{asignatura}' es: {notas[asignatura]}")

except KeyError:
    # Capturo una clave inexistente
    print(f"Asignatura incorrecta: {asignatura}")
```

Introduce una asignatura: Redes

La nota de la asignatura 'Redes' es: 5.2

5.10 Ejercicio 10

Construir una **list comprehension** con los números enteros positivos de una lista.

```
[228]: # Defino una lista con diferentes tipos de números
lista = [1, 4, -3, -1.5, 6.5, 2, 8, 2.1]

# Obtengo únicamente los enteros positivos
resultado = [numero for numero in lista if type(numero) == int and numero > 0]

# Muestro el resultado
print(resultado)
```

[1, 4, 2, 8]

5.11 Ejercicio 11

Construir una **set comprehension** que contenga únicamente las vocales presentes en una palabra.

```
[229]: # Defino una palabra
palabra = "murcielago"

# Obtengo las vocales sin repetir
resultado = {letra for letra in palabra if letra in ('a', 'e', 'i', 'o', 'u')}

# Muestro el conjunto
```

```
print(resultado)
```

```
{'a', 'i', 'e', 'o', 'u'}
```

5.12 Ejercicio 12

Construir una **list comprehension** con todos los números del 0 al 50 que contengan el dígito 3.

```
[230]: # Obtengo los números que contienen el dígito 3
resultado = [numero for numero in range(51) if '3' in str(numero)]

# Muestro la lista
print(resultado)
```

```
[3, 13, 23, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 43]
```

5.13 Ejercicio 13

Construir una **dictionary comprehension** donde cada palabra de una frase sea la clave y su longitud el valor.

```
[231]: # Defino una frase
frase = "Soy un ser humano"

# Creo el diccionario con la longitud de cada palabra
resultado = {palabra: len(palabra) for palabra in frase.split()}

# Muestro el diccionario
print(resultado)
```

```
{'Soy': 3, 'un': 2, 'ser': 3, 'humano': 6}
```

5.14 Ejercicio 14

Construir una **list comprehension** donde los números del 1 al 5 permanezcan como enteros y del 6 al 10 se muestren escritos en texto.

```
[232]: # Defino el diccionario de conversión
numero_a_palabra = {
    6: "seis",
    7: "siete",
    8: "ocho",
    9: "nueve",
    10: "diez"
}

# Construyo la lista solicitada
resultado = [
    numero if numero <= 5 else numero_a_palabra[numero]
    for numero in range(1, 11)
]
```



```
]
```

```
# Muestro el resultado
```

```
print(resultado)
```

```
[1, 2, 3, 4, 5, 'seis', 'siete', 'ocho', 'nueve', 'diez']
```